

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FALTY OF COMPUTER SCIENCE AND ENGINEERING



CAPSTONE PROJECT COMPUTER ENGINEERING

Motion Planning around Obstacles

Semester 251

Instructor: Assoc. Prof. Le Hong Trang

STT	Full name	ID	Note
1	Nguyen Tran Dang Khoa	2211635	
2	Ho Dang Khoa	2211588	

Ho Chi Minh City, December 2025

Tóm tắt

Hoạch định chuyển động cho robot trong môi trường chứa vật cản vốn là một bài toán tối ưu hóa phi tuyến đầy thách thức do tính chất không lồi của không gian cấu hình. Trong khi các phương pháp dựa trên lấy mẫu truyền thống thường gặp hạn chế về tính tối ưu và khả năng xử lý các ràng buộc động lực học phức tạp, bài báo này đề xuất một phương pháp tiếp cận mới dựa trên Đồ thị các Tập Lồi (Graphs of Convex Sets - GCS). Bằng cách phân rã không gian tự do thành các vùng lồi hữu hạn và mô hình hóa quỹ đạo robot sử dụng đường cong Bézier, chúng tôi thiết lập bài toán dưới dạng tối ưu hóa hỗn hợp nguyên. Đóng góp của phương pháp là việc áp dụng kỹ thuật nới lỏng lồi (convex relaxation) chặt chẽ để chuyển đổi bài toán phức tạp này thành bài toán Lập trình nón bậc hai (Second-Order Cone Programming - SOCP) dễ giải. Kết quả thực nghiệm trên các hệ thống có số chiều cao chứng minh phương pháp này vượt trội hơn so với các thuật toán PRM truyền thống, giảm thiểu đáng kể thời gian tính toán trong khi vẫn đảm bảo tìm được quỹ đạo tối ưu toàn cục. Chúng tôi kiểm chứng GCS trên một loạt các môi trường với độ phức tạp tăng dần, từ các vật cản 2D đơn giản và mê cung phức tạp đến các thiết bị bay quadrotor động lực học và tay máy 7 bậc tự do (7-DoF) trong không gian nhiều chiều. Các thực nghiệm số chứng minh rằng GCS đạt hiệu năng vượt trội một cách nhất quán so với các bộ hoạch định dựa trên lấy mẫu được sử dụng rộng rãi (ví dụ: PRM). Cụ thể, trong các nhiệm vụ nhiều chiều, GCS giảm thời gian truy vấn trực tuyến lên đến một bậc so với PRM tiêu chuẩn, đồng thời tìm ra các quỹ đạo tối ưu toàn cục ngắn hơn đáng kể (giảm tới 40% độ dài) so với các bộ hoạch định dựa trên lấy mẫu đã được xử lý hậu kỳ [1].

Contents

1	Introduction	1
1.1	Động lực và thách thức kỹ thuật	1
2	Theoretical Background	2
2.1	Graphs Theoretical	2
2.2	Kinodynamic Trajectory Generation	2
2.2.1	Kinodynamic Formulation	2
2.2.2	Smoothness and Continuity Constraints	3
2.2.3	Background on Bézier Curves	4
2.3	Convex Decomposition	6
2.3.1	Definition and Properties	6
2.3.2	Algorithms for Decomposition	6
2.3.3	Applications in Physics Engines	7
3	Related works	8
3.1	Sampling-Based Methods	8
3.2	Discussion	8
4	Proposed Solution	9
4.1	Phân hoạch không gian an toàn	9
4.2	Graphs of convex sets	10
4.3	Proposed solution	10
4.4	Experiment setup	10
4.4.1	Implementation details	10

4.5	Results and analysis	10
5	Conclusion	11
5.1	Summary	11
5.2	Future Work	11
	References	12
	Appendix A ...	13
A.1		13
A.2		13

List of Figures

Figure 2.1	Visual comparison between a concave mesh and its convex decomposition.	7
------------	--	---

List of Tables

Chapter 1

Introduction

In chapter 1, the overview, objectives and goals of the research's project are illustrated. The outline of the report is also presented.

1.1 Động lực và thách thức kỹ thuật

Chapter 2

Theoretical Background

In Chapter 2, it presents about preliminary knowledge in this project.

2.1 Graphs Theoretical

2.2 Kinodynamic Trajectory Generation

This section outlines the mathematical formulation for generating kinodynamically feasible trajectories. Unlike geometric path planning, which solely considers collision-free configurations in the workspace, motion planning (kinodynamic) accounts for the system's differential constraints, such as velocity, acceleration, and jerk limits. We leverage Bézier curves as the trajectory parameterization scheme, which is particularly advantageous for the Graph of Convex Sets (GCS) framework due to its convexity properties.

2.2.1 Kinodynamic Formulation

Consider a robotic system whose state evolution is governed by a set of ordinary differential equations (ODEs):

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(t) \in \mathcal{X}, \mathbf{u}(t) \in \mathcal{U} \quad (2.1)$$

where $\mathbf{x}(t) \in \mathbb{R}^n$ represents the state vector (typically comprising position, velocity, and acceleration), and $\mathbf{u}(t) \in \mathbb{R}^m$ denotes the control inputs. The goal is to compute a trajectory $\sigma(t) : [0, T] \rightarrow \mathcal{X}$ that minimizes a specific cost functional while satisfying:

1. **Boundary Conditions:** The start and goal states $\mathbf{x}(0) = \mathbf{x}_{start}$ and $\mathbf{x}(T) = \mathbf{x}_{goal}$.
2. **Safety Constraints:** $\sigma(t) \in \mathcal{X}_{free}$ for all t , ensuring collision avoidance.
3. **Dynamic Limits:** Inequality constraints on higher-order derivatives to ensure physical feasibility:

$$|\dot{\sigma}(t)| \leq v_{max}, \quad |\ddot{\sigma}(t)| \leq a_{max}, \quad |\dddot{\sigma}(t)| \leq j_{max} \quad (2.2)$$

2.2.2 Smoothness and Continuity Constraints

In the GCS framework, a full trajectory is composed of multiple piecewise Bézier segments. To ensure smooth motion across the boundaries of adjacent convex sets, continuity constraints must be enforced at the transition points. Let $r_A(t)$ defined by control points $\{a_0, \dots, a_n\}$ and $r_B(t)$ defined by $\{b_0, \dots, b_n\}$ be two consecutive segments.

- **C^0 Continuity (Position):** Ensures the path is connected.

$$a_n = b_0 \quad (2.3)$$

- **C^1 Continuity (Velocity):** Ensures continuous velocity, preventing infinite acceleration spikes.

$$a_n - a_{n-1} = b_1 - b_0 \quad (2.4)$$

- **C^2 Continuity (Acceleration):** Ensures continuous force/torque application, which is crucial for minimizing mechanical wear and jerk.

$$(a_n - 2a_{n-1} + a_{n-2}) = (b_2 - 2b_1 + b_0) \quad (2.5)$$

By enforcing these equality constraints at the edges of the graph, we generate a globally smooth trajectory that respects the underlying physics of the robot.

2.2.3 Background on Bézier Curves

In order to tackle the kinodynamic planning problem numerically, it is necessary to parameterize the trajectory functions through a finite number of decision variables. To this end, we employ Bézier curves. This section recalls the definition and the basic properties of this family of curves.

A Bézier curve is constructed using Bernstein polynomials. The k -th Bernstein polynomial of degree d , with $k = 0, \dots, d$, is defined as:

$$\beta_{k,d}(s) := \binom{d}{k} s^k (1-s)^{d-k}, \quad s \in [0, 1] \quad (2.6)$$

Note that the Bernstein polynomials of degree d are nonnegative and, by the binomial theorem, they sum up to one. Therefore, for each fixed $s \in [0, 1]$, the scalars $\{\beta_{k,d}(s)\}_{k=0}^d$ can be thought of as the coefficients of a convex combination. Bézier curves are obtained using these coefficients to combine a given set of $d+1$ control points $\gamma_k \in \mathbb{R}^n$:

$$\gamma(s) := \sum_{k=0}^d \beta_{k,d}(s) \gamma_k \quad (2.7)$$

It is easily verified that Bézier curves enjoy the following properties, which are instrumental for the GCS framework:

- **Endpoint values:** The curve γ starts at the first control point and

ends at the last control point:

$$\gamma(0) = \gamma_0 \quad \text{and} \quad \gamma(1) = \gamma_d \quad (2.8)$$

This property is essential for enforcing continuity constraints (C^0) when stitching multiple Bézier segments together in the graph.

- **Convex hull property:** The curve γ is entirely contained in the convex hull of its control points:

$$\gamma(s) \in \text{conv}(\{\gamma_k\}_{k=0}^d), \quad \forall s \in [0, 1] \quad (2.9)$$

In the context of GCS, if a convex region X_v represents a safe set, guaranteeing collision avoidance is reduced to constraining all control points to lie within that set:

$$\gamma_k \in X_v, \quad \forall k \in \{0, \dots, d\} \implies \gamma(s) \in X_v \quad (2.10)$$

This allows us to enforce continuous safety constraints through a finite set of linear inclusion constraints.

- **Derivative (Hodograph):** The derivative $\dot{\gamma}$ of the curve γ is also a Bézier curve, but of degree $d-1$, with control points defined by the difference of adjacent points:

$$\dot{\gamma}(s) = \sum_{k=0}^{d-1} \beta_{k,d-1}(s) \dot{\gamma}_k \quad (2.11)$$

where $\dot{\gamma}_k = d(\gamma_{k+1} - \gamma_k)$ for $k = 0, \dots, d-1$. Similarly, higher-order derivatives (acceleration, jerk) can be expressed as linear combinations of the control points. Consequently, kinodynamic constraints (e.g., velocity limits $|\dot{\gamma}(s)| \leq v_{max}$) can be imposed as linear constraints on the differences between adjacent control points, preserving the convexity

of the optimization problem.

- **Integral of convex function:** For a convex function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ (often representing the cost function in motion planning, such as energy or path length), the integral along the curve is bounded by the convex combination of the function values at the control points:

$$\int_0^1 f(\gamma(s))ds \leq \frac{1}{d+1} \sum_{k=0}^d f(\gamma_k) \quad (2.12)$$

This inequality provides a convex upper bound on the integral cost, facilitating efficient optimization within the GCS mixed-integer convex programming formulation.

2.3 Convex Decomposition

This section provides an overview of convex decomposition, a fundamental technique used in computational geometry and physics simulations.

2.3.1 Definition and Properties

Convex decomposition involves breaking down a concave polygon or polyhedron into a set of convex sub-shapes. A shape is defined as convex if, for every pair of points within the shape, every point on the straight line segment that joins the pair of points is also within the shape.

$$\forall x, y \in S, \forall \lambda \in [0, 1], \lambda x + (1 - \lambda)y \in S \quad (2.13)$$

2.3.2 Algorithms for Decomposition

Several algorithms exist for performing convex decomposition, varying in complexity and the optimality of the resulting set of convex hulls.

- **Exact Convex Decomposition:** Produces the minimum number of convex pieces but is often computationally expensive (NP-hard for general 3D shapes).
- **Approximate Convex Decomposition (ACD):** Prioritizes speed and simpler geometry by allowing a small amount of concavity or error, which is often sufficient for collision detection.

2.3.3 Applications in Physics Engines

In the context of this project, convex decomposition is critical for efficient collision detection. Physics engines like PhysX or Bullet perform collision checks much faster between convex shapes than concave ones.

Figure 2.1: Visual comparison between a concave mesh and its convex decomposition.

Chapter 3

Related works

...

3.1 Sampling-Based Methods

3.2 Discussion

In this chapter, we have seen three distinct approaches to enhance...

Chapter 4

Proposed Solution

...

4.1 Phân hoạch không gian an toàn

Thay vì tiếp cận bài toán tránh vật cản theo cách truyền thống là mô tả các ràng buộc không lỗi dựa trên bề mặt vật cản, nhóm tác giả đề xuất một phương pháp chuyển đổi không gian cấu hình tự do (collision-free configuration space) thành hợp của một tập hợp hữu hạn các vùng lỗi bị chặn (bounded convex regions). Trong mô hình này, các vùng lỗi có thể chồng lấn lên nhau, và bài toán hoạch định chuyển động được phát biểu lại dưới dạng một chương trình quy hoạch lồi nguyên hỗn hợp (Mixed-Integer Convex Program - MICP). Cụ thể, các biến nguyên được sử dụng để gán từng đoạn quỹ đạo đa thức (polynomial trajectory segments) vào các vùng lỗi an toàn cụ thể này. Phương pháp này mang lại lợi thế lớn về mặt tính toán vì số lượng biến nguyên chỉ phụ thuộc vào số lượng vùng an toàn được tạo ra (sử dụng thuật toán IRIS) thay vì phụ thuộc vào số lượng mặt của vật cản, giúp giải quyết hiệu quả các môi trường phức tạp với hàng trăm vật cản. Hơn nữa, việc ràng buộc quỹ đạo nằm trọn trong các tập lỗi đảm bảo an toàn tuyệt đối cho toàn bộ hành trình liên tục, khắc phục nhược điểm "cắt góc" (corner cutting) thường gặp ở các phương pháp chỉ kiểm tra

va chạm tại các điểm lấy mẫu rời rạc.

4.2 Graphs of convex sets

Cho một đồ thị vô hướng $G = (V, E)$ với tập đỉnh $V = \{1, 2, \dots, n\}$ và tập cạnh $E \subseteq V \times V$. Mỗi đỉnh $v \in V$

4.3 Proposed solution

4.4 Experiment setup

4.4.1 Implementation details

4.5 Results and analysis

Chapter 5

Conclusion

5.1 Summary

5.2 Future Work

Future research will focus on...

- ...
- ...

References

- [1] R. Deits and R. Tedrake, “Motion planning around obstacles with convex optimization,” *arXiv preprint arXiv:2205.04422*, 2022.
- [2] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004, ISBN: 0521833787.
- [3] L. A. Wolsey, *Integer Programming*. Wiley-Interscience, 1999, ISBN: 0471283665.
- [4] M. Conforti, G. Cornuéjols, and G. Zambelli, *Integer Programming*. Springer, 2014, ISBN: 9783319110073.
- [5] M. Lubin, E. Yamangil, R. Bent, and J. P. Vielma, “Polyhedral approximation in mixed-integer convex optimization,” *Mathematical Programming*, vol. 172, pp. 139–168, 2018.

Appendix A

...

A.1 ...

A.2 ...